Exercise 1: Inventory Management System

1. Understand the Problem

Why Data Structures and Algorithms are Essential

In a warehouse inventory system, thousands of products may be stored. Efficient data structures and algorithms help in:

- Fast product insertion and retrieval.
- Quick inventory updates.
- Efficient storage utilization.
- Reduced processing time for large inventories.
- Better scalability as the inventory grows.

Without proper data structures, searching or updating products would become slow and inefficient.

Some suitable data structures are:

- ArrayList – Easy to implement but searching is slower.
- HashMap – Provides fast insertion, deletion, and lookup using product IDs.
- Linked List – Useful when frequent insertions and deletions are required.
- TreeMap – Maintains sorted order of products.

For this implementation, HashMap is preferred because it offers near constant-time access.

2. Setup

```
import java.util.*;
class Product {
 int productId;
 String productName;
 int quantity;
 double price;

 Product(int productId, String productName, int quantity, double price) {
  this.productId = productId;
  this.productName = productName;
  this.quantity = quantity;
  this.price = price;
 }
```

```
 public String toString() {
  return "ID: " + productId + " Name: " + productName + " Quantity: " + quantity
+ " Price: " + price;
   }
}
```

3. Implementation

```
class Inventory {
 HashMap<Integer, Product> products = new HashMap<>();

 public void add(Product product) {
  products.put(product.productId, product);
  System.out.println("Product added");
 }

 public void update(int productId, int quantity, double price) {
  if (products.containsKey(productId)) {
   Product p = products.get(productId);
   p.quantity = quantity;
   p.price = price;
   System.out.println("Product Updated");
  } else System.out.println("Product Not Found");
 }

 public void delete(int productId) {
  if (products.remove(productId) != null) System.out.println("Product Deleted");
  else System.out.println("Product Not Found");
 }
```

```
 public void displayProducts() {
  for(Product p : products.values()) {
   System.out.println(p);
  }
 }
}
```

```
public class InventoryManagement {
 public static void main(String[] args) {
  Inventory inv = new Inventory();
```

```
  inv.add(new Product(101, "Laptop", 5, 100000));
  inv.add(new Product(102, "Mouse", 50, 5000));
```

```
    inv.displayProducts();
    inv.update(101, 2, 35000);

    inv.delete(102);
    inv.displayProducts();
  }
}
```

4. Analysis

Operation	***HashMap Complexity***
Add Product	O(1)
Update Product	O(1)
Delete Product	O(1)
Search Product	O(1)

Optimization

- Use HashMap instead of arrays for faster lookups.
- Use productId as the key.
- Resize HashMap automatically for large inventories.
- Use caching for frequently accessed products.